

Programming Assignment 3

Sanoj S Vijendra
22b0916

Table of Content

- 1. Introduction**
- 2. Code Overview**
- 3. Algorithm**
 - a. Alpha Beta pruning**
 - b. Transposition table**
 - c. Move Ordering**
- 4. Heuristics**
 - a. Position Assessment Heuristics**
 - i. Material Heuristics**
 - ii. Positional Heuristics**
 - b. Move Ranking Heuristics**
 - i. Promotions**
 - ii. Captures**
 - iii. Quite moves**
- 5. Key Observations**
- 6. Conclusion**

Introduction

Assignment's aim was to develop an agent which is capable of playing microchess (a variant of chess with 5x4 board) against both rational and random players. In total we were supposed to create three agents which are capable of playing against the players of different difficulty (random being easiest and rational being hardest).

The most important aspect was the tradeoff between searching multiple moves ahead for a perfect move and time constraints. The initial approach was the Minimax algorithm, which assumes the opponent will always play their best move. This agent's "smarts" came from an evaluation function that assigned a numerical score to any board state based on material (the value of pieces) and positional heuristics (like pawn advancement and central piece control).

But this implementation was too slow. So, finally alpha-beta pruning to prune the bad branches and maintaining a transposition table to remember and reuse the states already seen were included. This helped to increase the speed pretty much.

Code Overview

Following are the different functions involved in the implementation:

1. **move(self, ...)**: Set the desired search depth and call search function (find_best_move) to get the result.
2. **evaluate_board(self, ...)**: This is the brain of our agent. Its job is to look at a single, non-final board position and assign it a numerical score (e.g., "+150" means "White is slightly ahead").
3. **move_score(self, ...)**: This helper function is used for optimisation. It guesses if a move is good or bad without searching.
4. **minimax(self, ...)**: The core of our agent. It's a recursive function that dives deep into the game tree, exploring all possible scenarios.
5. **find_best_move(self, ...)**: This function starts the search. It's the non-recursive head that loops through the first set of legal moves and calls minimax for each one to see which is best.
6. **reset(self, ...)**: This function is called between games to clear the transposition table (self.trans_tbl) so that memories from the last game don't interfere with the new one.

Algorithm

Minimax with alpha beta pruning is used.

1. *move* function calls *find_best_move*, which lists all the valid moves, which calls *minimax* which checks the worst case score if a move is made.
2. *minimax* simulates **Maximising player** (our agent (wanting highest score)) against the **minimising player** (opponent (wanting lowest score)). Recursion stops if one of the three happens.
 - a. **Game Over**: Checks *chess_obj.game_result()*. A win is a huge positive score (+1,000,000), a loss is a huge negative (-1,000,000), and a draw is 0.
 - b. **Depth Limit**: If `depth == 0`, we've searched as far as we're allowed. The agent calls *evaluate_board* to get a best guess score for this position.
 - c. **Stalemate**: if no *legal_moves* left, then this is a draw, so it returns 0.
3. **Alpha-Beta pruning** and **Transposition table** are used to make our agent fast and optimised.
 - a. **Alpha-Beta Pruning**: This is the *if(beta <= alpha): break* logic. It's a shortcut that stops the search early. It works like this:
 - i. Imagine if our agent finds a move (Move A) that guarantees a score of +100.
 - ii. It then starts analysing Move B. On the opponent's first reply, the opponent finds a move that leads to a score of +50.
 - iii. Our agent will immediately stop searching Move B's entire branch because the opponent (a perfect "Minimizer") will force the +50 score, which is worse than the +100 score we are already guaranteed from Move A. This prunes the branch and saves massive amounts of time.
 - b. **Transposition table**: This acts as our agent's memory during the game.
 - Problem: We can reach the exact same board position through different move orders (e.g., A -> B then C -> D vs. C -> D then A -> B).
 - Solution: Every time minimax finds a score for a position, it stores it in *self.trans_tbl* using the board state (bitboards) and depth as a key. The very first thing the function does is check this table. If the key exists, it instantly returns the stored score instead of re-calculating it.
 - c. **Move Ordering** (*self.move_score*): This makes Alpha-Beta Pruning much more effective. Pruning works best if we check the best moves first.

- i. Before starting the minimax loop, we call *sorted_moves* = *sorted(...)*.
 - ii. It uses the *move_score* function to guess which moves are best (e.g., captures, promotions).
 - iii. By checking good moves first, it finds a strong alpha or beta value early, which allows it to prune more branches more aggressively.
4. Finally, *find_best_move* collects all the scores from these top level calls, picks the one with the best score (highest if White, lowest if Black), and returns that move.

Heuristics

Our agent uses two different heuristics, one for ranking moves and one for accessing position.

1. **Position Assessment Heuristics:** It is used when the search depth limit is reached and the agent must make a final guess on a position's value. The score is a sum of two components:
 - a. **Material Heuristic:** The agent simply sums the value of all pieces on the board based on our *self.piece_val* dictionary:
 - Pawn: 100
 - Knight: 320
 - Bishop: 330
 - Rook: 500
 - Queen: 900
 - King: 20,000 (A high value)

The final score is the difference of our materials and opponent's materials.
 - b. **Positional Heuristic:** It recognises that a piece's location matters.
 - i. Pawn: The tables *self.pawn_w* and *self.pawn_b* awards bonus for pawns as they advance. For example, *self.pawn_w* gives +10 on row 3 and +30 on row 1, encouraging the agent to push pawns toward promotion.
 - ii. Knight: *self.knight* table gives bonuses for being in the center. A knight on (2, 2) gets +4, while one on (0, 0) gets +1.
 - iii. Bishop: Similar to the knight, *self.bishop* gives a small bonus for being active in the center.
2. **Move Ranking Heuristics (in *move_score*):** It ranks moves based on these rules, in this order:
 - a. **Promotions (Highest Priority):** Any move that results in a pawn promotion (*promo* != -1) is considered the best. It is given a score of 1000 + *value_of_promoted_piece*.

- b. **Captures (Medium Priority):** Any move that captures an opponent's piece (*victim* $\neq -1$). It's given a score of $500 + \text{value_of_victim}$. This means it prioritises capturing a Queen over a Pawn.
- c. **Quiet Moves (Lowest Priority):** Any other move (like just moving a knight to an empty square) is given a score of 0 and checked last.

Key Observations

I used same algorithm across the three tasks with the only difference being the search depth, 2 in task1, 4 in task2 and 5 in task3.

According to the autograder, the following results were achieved:

1. **Task1 (against random agent):**
 - a. Win: 64, Loss: 0, Draw: 36.
 - b. Avg. time per move: 0.003552 sec
2. **Task2 (against rational agent with max depth 4):**
 - a. Win: 43, Loss: 8, Draw: 49
 - b. Avg. time per move: 0.040442 sec
3. **Task3 (against rational agent with max depth 5):**
 - a. Win: 66, Loss: 1, Draw: 33
 - b. Avg. time per move: 0.155240 sec

As we can see we see that the average time per move is increasing as we are increasing the depth of our agent with 11x more from task1 to task2 and 3.8x from task2 to task3. This is because as depth increases the search space increases and hence more time is spent searching for the best move.

Conclusion

The desired behaviour of our chess agent was achieved using the minimax algorithm with alpha beta pruning, memory table(transposition table) and move ordering. These three really helped in speeding up the process. Overall the agent showed the win:loss ratio of 173:9 which I think is really good.